



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

CH Yip
May 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

Summary of Methodologies

- This project employed an end-to-end data science pipeline to predict whether SpaceX's Falcon 9 first stage booster would land successfully. Data was collected programmatically via HTTP GET requests to the SpaceX REST API, extracting launch records covering booster versions, payload mass, orbital targets, launchpad coordinates, and core landing outcomes. The raw JSON data was parsed and structured into a Pandas DataFrame, with missing values identified and handled accordingly. Data wrangling then converted multi-class landing outcome descriptors into a clean binary classification label — success (1) or failure (0) — enabling supervised machine learning. Exploratory data analysis was subsequently conducted using Matplotlib and Seaborn to uncover statistical correlations between launch variables and landing success, followed by interactive geospatial analysis using Folium, where marker clustering, color-coded outcome indicators, and proximity distance calculations were applied to reveal geographical patterns across all four launch sites.

Summary of Results

- The analysis revealed that all four SpaceX launch sites — KSC LC-39A, CCAFS LC-40, CCAFS SLC-40, and VAFB SLC-4E — are deliberately positioned at low latitudes along coastlines to maximize rotational velocity boost and ensure safe overwater ascent trajectories. The Florida cluster at Cape Canaveral (~28.5°N) serves equatorial and geosynchronous orbits, while Vandenberg (~34.6°N) supports polar missions. Landing success rates varied across sites and improved notably over time as SpaceX refined its reusability technology, with drone ship (ASDS) and ground pad (RTLS) landings becoming increasingly reliable in later launches. The binary classification label derived during data wrangling showed a mean success rate that underscored SpaceX's growing proficiency in booster recovery — a key factor behind its ability to offer launch costs significantly below the industry average of \$165 million, compared to its advertised price of \$62 million per Falcon 9 launch.

Introduction

- **Project Background:** The commercial space industry has changed dramatically over the past decade, with private companies driving down costs and opening up access to space in ways that were previously unimaginable. Among these, SpaceX stands out for its track record — resupplying the International Space Station, launching the Starlink satellite internet network, and sending astronauts to space — all while keeping launch costs remarkably competitive. A Falcon 9 launch runs around \$62 million, a fraction of the \$165 million or more that rivals typically charge. Much of that saving comes down to one thing: the ability to land and reuse the rocket's first stage, which does the heavy lifting during launch. That said, a successful landing isn't a given — mission demands around payload weight, target orbit, and customer needs sometimes mean the first stage is expended rather than recovered. This project sits at the intersection of that engineering and economics challenge.
- **Problem:** Playing the role of a data scientist at a fictional startup called Space Y, the goal is to use publicly available SpaceX data and machine learning to predict whether the first stage will be recovered after a given launch — and by extension, get a clearer picture of what that launch actually costs.



Section 1

Methodology

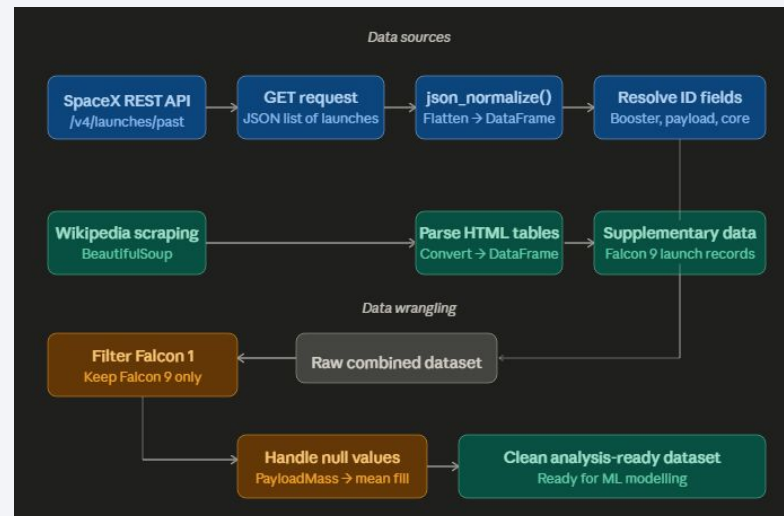
Methodology

Executive Summary

- Data collection methodology:
 - Data was collected by making HTTP GET requests to the SpaceX REST API (api.spacexdata.com/v4), retrieving detailed launch records including booster versions, launchpad coordinates, payload mass, orbital targets, and core landing outcomes through a series of purpose-built helper functions. The raw JSON responses were then parsed, cleaned, and structured into a Pandas DataFrame — with missing values handled and data types standardized — to prepare a well-formatted dataset ready for downstream exploratory analysis and machine learning.
- Perform data wrangling
 - Data wrangling involved performing exploratory analysis to identify and handle missing values across all attributes, categorize numerical and categorical columns, and examine the distribution of launches across sites and orbit types using Pandas `value_counts()`. The key transformation was converting the raw multi-class landing Outcome column — which contained descriptive strings like True ASDS, False RTLS, and None None — into a clean binary training label (`landing_class`), where 1 indicates a successful booster landing and 0 indicates failure, making the dataset ready for supervised machine learning.
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models

Data Collection

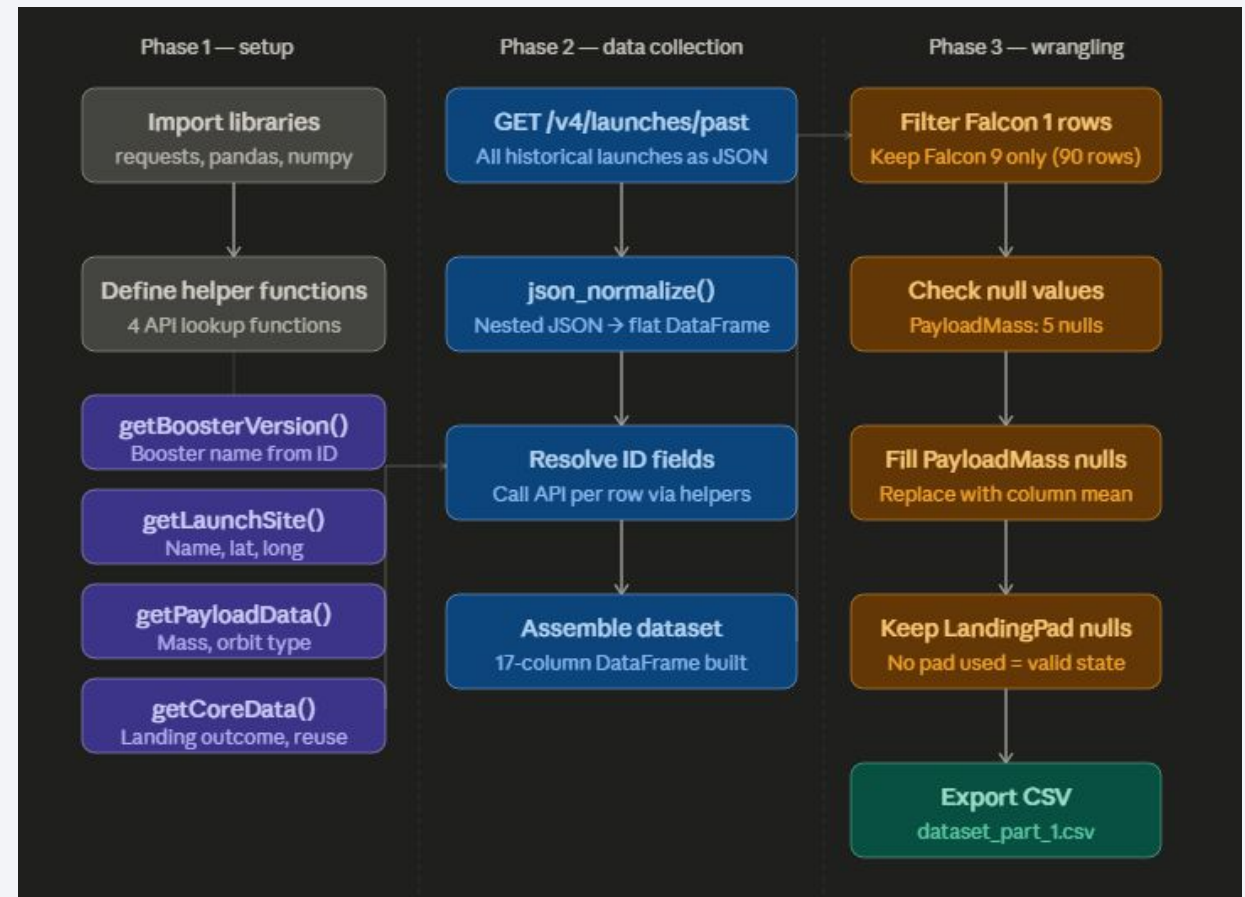
- The dataset underpinning this project was pulled together from two main sources: the **SpaceX REST API** and **Wikipedia web scraping**. Rather than relying on a pre-packaged dataset, the data had to be actively retrieved, cleaned, and shaped into something workable — which is pretty much how it goes in real data science projects.
 - **Source 1 — SpaceX REST API**
 - The primary source of launch data was the SpaceX REST API, accessed through the base URL `api.spacexdata.com/v4/`, with the `/launches/past` endpoint used to pull historical launch records. A GET request via Python's `requests` library returned a JSON response — essentially a list of objects, each representing a single launch event.
 - From there, the `json_normalize()` function flattened the nested JSON into a readable Pandas DataFrame. However, several columns — like `rocket` and `launchpad` — returned only ID numbers rather than descriptive values. A second round of API calls was necessary, targeting endpoints for boosters, launchpads, payloads, and cores, to swap those IDs out for actual data.
 - **Source 2 — Wikipedia Web Scraping**
 - As a supplementary source, Falcon 9 launch records were scraped from relevant Wikipedia pages using Python's `BeautifulSoup` package. HTML tables were parsed and converted directly into Pandas DataFrames, providing an additional layer of structured launch history to work with.



Data Collection – SpaceX API

- This notebook is Lab 1 of the SpaceX Falcon 9 Landing Prediction capstone. Its sole purpose is to build a clean, analysis-ready dataset from scratch using the SpaceX REST API. The work falls into three clear phases: setting up helper functions, pulling and enriching raw launch data, and wrangling the result into a usable table.
- It starts by importing `requests`, `pandas`, `numpy`, and `datetime`, then defines four helper functions — `getBoosterVersion`, `getLaunchSite`, `getPayloadData`, and `getCoreData` — each of which hits a different API endpoint to swap ID references in the raw data for real descriptive values. The main data pull targets `/v4/launches/past`, which returns all historical launches as a JSON list, flattened into a DataFrame via `json_normalize`. Falcon 1 records are then filtered out, leaving 90 Falcon 9 flights. Finally, missing `PayloadMass` values are filled with the column mean, while `LandingPad` nulls are left intentionally (they indicate no pad was used). The output is exported as `dataset_part_1.csv`.

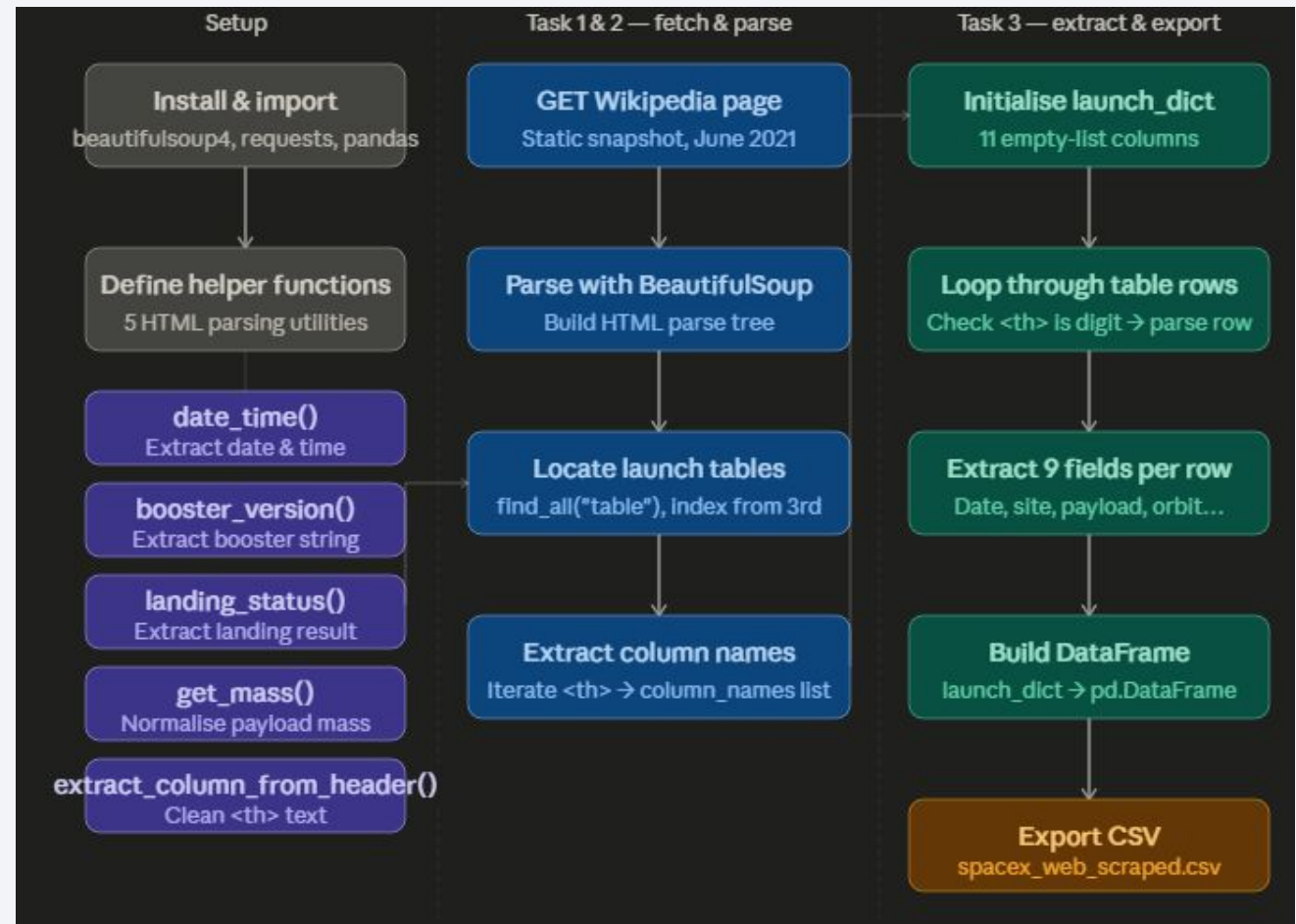
- 1.jupyter-labs-spacex-data-collection-api.ipynb



Data Collection - Scraping

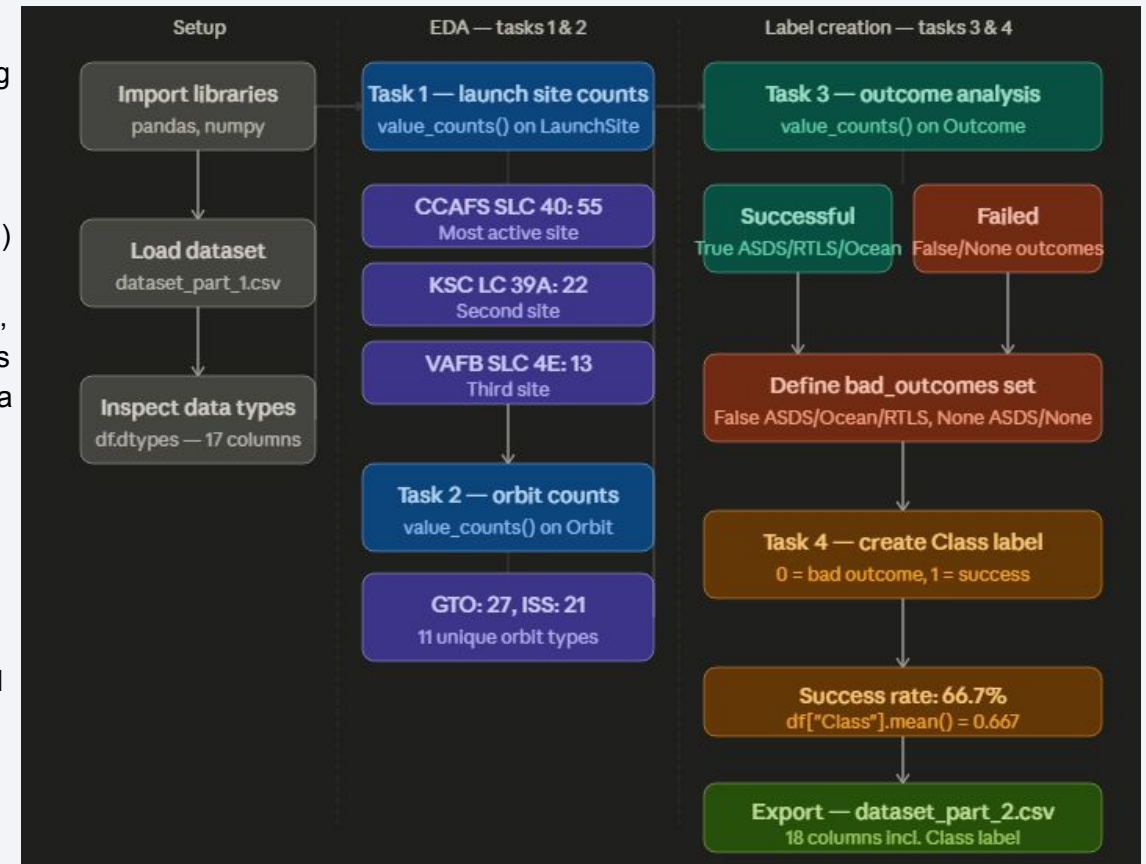
- This is Lab 2 of the SpaceX capstone, and it takes a different approach to data collection compared to the API lab — instead of querying a live endpoint, it scrapes a static snapshot of the Wikipedia page "List of Falcon 9 and Falcon Heavy launches" (archived at June 9, 2021) to extract structured launch records directly from HTML tables.
- The notebook is structured around three tasks. First, it uses `requests` to fetch the raw Wikipedia page HTML, then passes it to `BeautifulSoup` to create a navigable parse tree. Second, it locates the relevant launch tables (starting from the third table on the page), iterates through the `<th>` header elements, and uses the helper function `extract_column_from_header()` to pull out clean column names. Third, it loops through every table row, checks whether the row represents a valid flight entry (by testing if the row header is a digit), and extracts nine fields per row — flight number, date, time, booster version, launch site, payload, payload mass, orbit, customer, launch outcome, and booster landing — using dedicated helper functions for parsing noisy HTML cell content. The collected data is assembled into a Python dictionary, converted to a Pandas DataFrame, and exported as `spacex_web_scraped.csv`.

- 2.jupyter-labs-webscraping.ipynb



Data Wrangling

- This lab 3 takes the cleaned dataset from Lab 1 and prepares it for machine learning by running exploratory data analysis (EDA) and, crucially, engineering the target label that the model will eventually learn to predict.
 - The work is structured around four tasks. First, launch counts are tallied by site using `value_counts()` on the `LaunchSite` column — revealing that CCAFS SLC 40 handled the most missions (55), followed by KSC LC 39A (22) and VAFB SLC 4E (13). Second, orbit type frequencies are counted the same way, showing GTO (27) and ISS (21) as the most common destinations. Third, the raw `Outcome` column is examined — it contains eight distinct string values like `True ASDS`, `False Ocean`, `None None`, each describing both whether a landing was attempted and whether it succeeded.
 - The most important step is the fourth: creating the binary `Class` label. A set called `bad_outcomes` is defined to capture all unsuccessful or unattempted landings (`False ASDS`, `False Ocean`, `False RTLS`, `None ASDS`, `None None`). The dataset is then iterated row by row: if a row's `Outcome` falls in `bad_outcomes`, it gets a `Class` of 0 (failed); otherwise it gets a 1 (successful). The resulting column is appended to the DataFrame. The overall success rate works out to approximately 66.7%. The labelled dataset is then exported as `dataset_part_2.csv`, ready for the modelling pipeline.
- 3.labs-jupyter-spacex-Data wrangling.ipynb



EDA with SQL

Task 1 — Unique Launch Sites

- Retrieved all distinct launch sites from `SPACEXTABLE`
- Found 4 unique sites: `CCAFS LC-40`, `VAFB SLC-4E`, `KSC LC-39A`, `CCAFS SLC-40`

Task 2 — Filter by Site Prefix

- Filtered records where `Launch_Site` begins with `'CCA'`
- Returned the first 5 matching records using `.head()`

Task 3 — Total Payload Mass

- Summed `PAYLOAD_MASS__KG_` across all records
- Total payload carried: **619,967 kg**

Task 4 — Average Payload Mass

- Calculated the mean of `PAYLOAD_MASS__KG_` across all booster versions
- Average payload mass: **~6,138 kg**

Task 5 — First Launch Date

- Applied `MIN()` on the `Date` column to find the earliest record
- First date in the dataset: **2010-06-04**

Task 6 — Successful Missions with Payload Range

- Filtered rows where `Mission_Outcome = 'Success'` and `PAYLOAD_MASS__KG_` was between **4,000 and 6,000 kg**
- All 4 launch sites appeared in the results

Task 7 — Mission Outcome Count

- Separated records by `Mission_Outcome` to count successes vs failures
- Result: **98 successful** missions and **3 failed** missions

Task 8 — Max Payload Launch Sites

- Used a subquery with `MAX()` on `PAYLOAD_MASS__KG_` to identify which launch sites handled the heaviest payloads
- Sites: **CCAFS SLC-40** and **KSC LC-39A**

Task 9 — 2015 Drone Ship Landing Failures

- Filtered records where `Date LIKE '2015%'` and `Landing_Outcome = 'Failure (drone ship)'`
- Returned the date, booster version, and launch site for each matching record

Task 10 — Ranked Landing Outcomes (2010–2017)

- Grouped all records by `Landing_Outcome` within the date range `2010-06-04` to `2017-03-20`
- Applied `COUNT()` and sorted results in descending order using `ORDER BY COUNT DESC`

4.jupyter-labs-eda-sql-coursera_sqlite.ipynb

EDA with Data Visualization

Chart 1 — Flight Number vs. Payload Mass (scatter)

- **Type:** `sns.catplot` scatter
- **Why:** To get an early read on whether launch experience and payload weight jointly influence landing success. The chart showed that as flight numbers increase over time, successful landings become more frequent — suggesting SpaceX improved its landing capability with practice. Even heavier payloads showed successful recoveries in later flights.

Chart 2 (Task 1) — Flight Number vs. Launch Site (scatter)

- **Type:** `sns.catplot` scatter
- **Why:** To check whether certain launch sites were associated with more successful landings, and whether success improved over time at each site. This helped identify site-level performance trends across the mission timeline.

Chart 3 (Task 2) — Payload Mass vs. Launch Site (scatter)

- **Type:** `sns.catplot` scatter
- **Why:** To examine whether heavier payloads were only launched from specific sites, and whether payload mass affected success rates differently across sites. A key finding was that VAFB SLC 4E had no launches with payload mass above 10,000 kg, suggesting operational constraints at that site.

Chart 4 (Task 3) — Success Rate by Orbit Type (bar chart)

- **Type:** Bar chart
- **Why:** To compare landing success rates across the 11 orbit types in the dataset. A bar chart was the right choice here because it directly compares a single metric (success rate) across discrete categories (orbit types), making it easy to rank which orbits consistently produced successful landings vs. which ones were more hit-or-miss.

EDA with Data Visualization

Chart 5 (Task 4) — Flight Number vs. Orbit Type (scatter)

- **Type:** `sns.catplot` scatter
- **Why:** To see whether success within each orbit type improved as SpaceX accumulated more flights. The chart confirmed that LEO orbit success was closely tied to flight experience, while GTO showed no clear relationship between flight number and outcome — likely due to the greater energy demands of GTO missions making recovery harder regardless of experience.

Chart 6 (Task 5) — Payload Mass vs. Orbit Type (scatter)

- **Type:** `sns.catplot` scatter
- **Why:** To understand how payload mass interacted with orbit type in determining landing success. The result showed that heavier payloads in Polar, LEO, and ISS orbits still achieved successful landings, whereas GTO outcomes were harder to separate by mass alone — both successes and failures appeared across the payload range.

Chart 7 (Task 6) — Yearly Success Rate Trend (line chart)

- **Type:** Line chart
- **Why:** To track how SpaceX's landing success rate evolved year by year. A line chart was appropriate here because it communicates change over time clearly. The trend confirmed a steady rise in success rate from 2013 through to 2020, reflecting SpaceX's iterative improvements in booster recovery technology.

Taken together, the scatter plots mapped how individual variables related to landing outcome, while the bar chart ranked orbit types by success and the line chart captured the overall improvement over time. Together they provided the feature-level insights needed to select inputs for the machine learning model in the next stage.

Build an Interactive Map with Folium

Folium Map Objects — Summary

- This lab used the `folium` library to build an interactive map that explored the geographical context of SpaceX's launch sites across three tasks. Each layer of map objects added a new dimension of insight on top of the one before it.

Task 1 — Launch Site Circles & Text Markers

- **Objects added:** `folium.Circle` and `folium.map.Marker` (with `DivIcon` text labels) for each of the four launch sites.
- **Why:** Raw latitude and longitude numbers in a DataFrame are hard to interpret visually. Pinning each site as a labelled circle on an interactive map made it immediately clear where the sites are located geographically — revealing that all four are situated close to coastlines and near the equator. This is no accident; coastal proximity allows rocket debris to fall safely over water, and equatorial launches benefit from the Earth's rotational velocity, reducing the fuel needed to reach orbit.

Task 2 — Colour-coded Launch Outcome Markers with MarkerCluster

Objects added: `folium.Marker` with `folium.Icon` (green = success, red = failure) grouped inside a `MarkerCluster`.

Why: Many launches share the same coordinates since they depart from the same site, so stacking individual markers would clutter the map. `MarkerCluster` groups nearby markers into a single cluster that expands on zoom, keeping the map readable. The green/red colour coding mapped the `Class` label (1 = landing success, 0 = failure) directly onto the map, making it easy to spot which sites had higher success rates at a glance — without needing to look at a table or chart.

Task 3 — MousePosition, Distance Markers & PolyLines

Objects added:

- `MousePosition` plugin — displays live latitude/longitude coordinates as the mouse moves across the map
- `folium.Marker` with `DivIcon` distance labels — placed at the nearest coastline, highway, railway, and city to a launch site
- `folium.PolyLine` — straight lines connecting each launch site to its nearby landmarks

Why: The `MousePosition` plugin turned the map into an interactive measuring tool — by hovering over any point, the coordinates could be read off and fed into the Haversine `calculate_distance()` function to compute exact distances in kilometres. The `PolyLines` made the distances visible as straight lines on the map, and the `DivIcon` markers displayed the computed distance value directly at the endpoint. Together these objects answered four concrete geographical questions: how close each launch site is to the coastline, a highway, a railway, and the nearest city — helping explain why launch sites are sited where they are from a logistics and safety standpoint.

- 6.lab_jupyter_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

Interaction 1 — Launch Site Dropdown (Task 1)

- A `dcc.Dropdown` listing all four launch sites plus an "All Sites" option. Added to give users control over which site's data is displayed, enabling drill-down analysis without running separate queries.

Plot 1 — Success Pie Chart (Task 2)

- A `px.pie` chart driven by the dropdown via a Dash callback. On "All Sites," it shows successful launch counts broken down by site. On a specific site, it switches to a success vs. failure split. Pie charts were chosen because they communicate proportional composition clearly through colour-coded segments — making success rates instantly readable.

Interaction 2 — Payload Range Slider (Task 3)

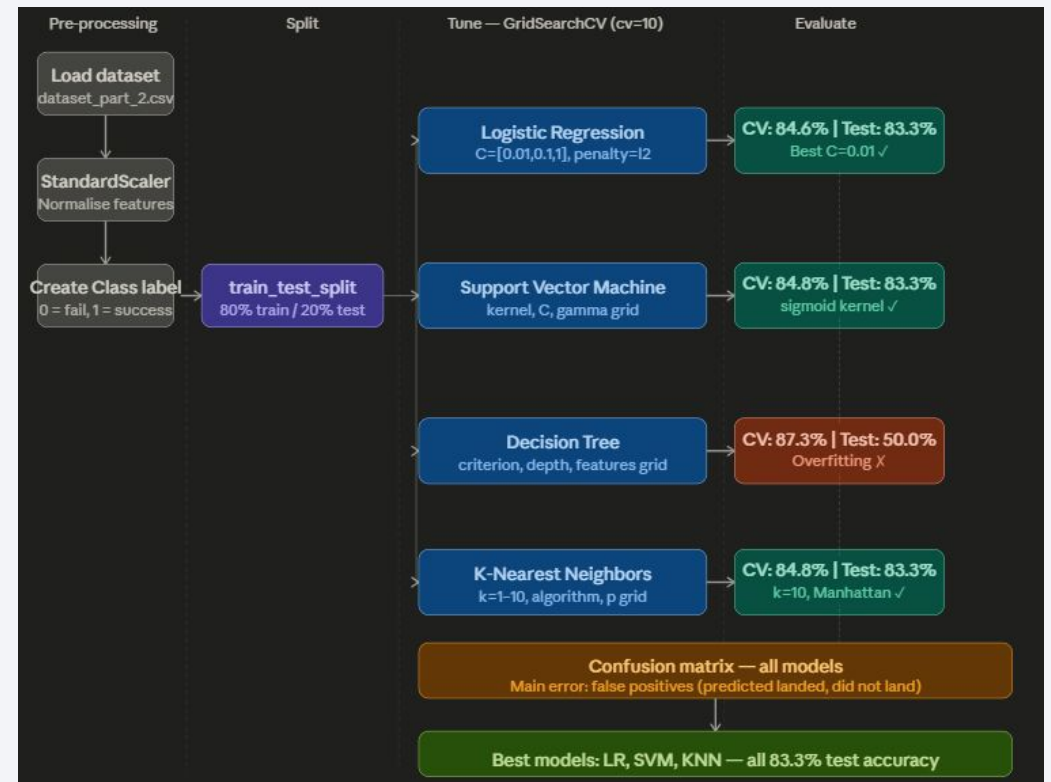
- A `dcc.RangeSlider` spanning 0–10,000 kg, letting users set a payload window. Added because payload mass is a key factor in landing success — heavier payloads leave less fuel for first stage recovery — so filtering by range reveals how that relationship plays out across different missions.

Plot 2 — Payload vs. Outcome Scatter Chart (Task 4)

- A `px.scatter` with payload mass on the x-axis, launch outcome on the y-axis, and points colour-coded by booster version. Responds to both the dropdown and slider simultaneously. A scatter plot was chosen to expose correlations between payload weight, booster type, and landing success — the dual interactivity lets users ask highly specific questions across site and payload combinations.
- 7.plotly dash - pie chart.png
 - 8.plotly dash - scatter plot.png

Predictive Analysis (Classification)

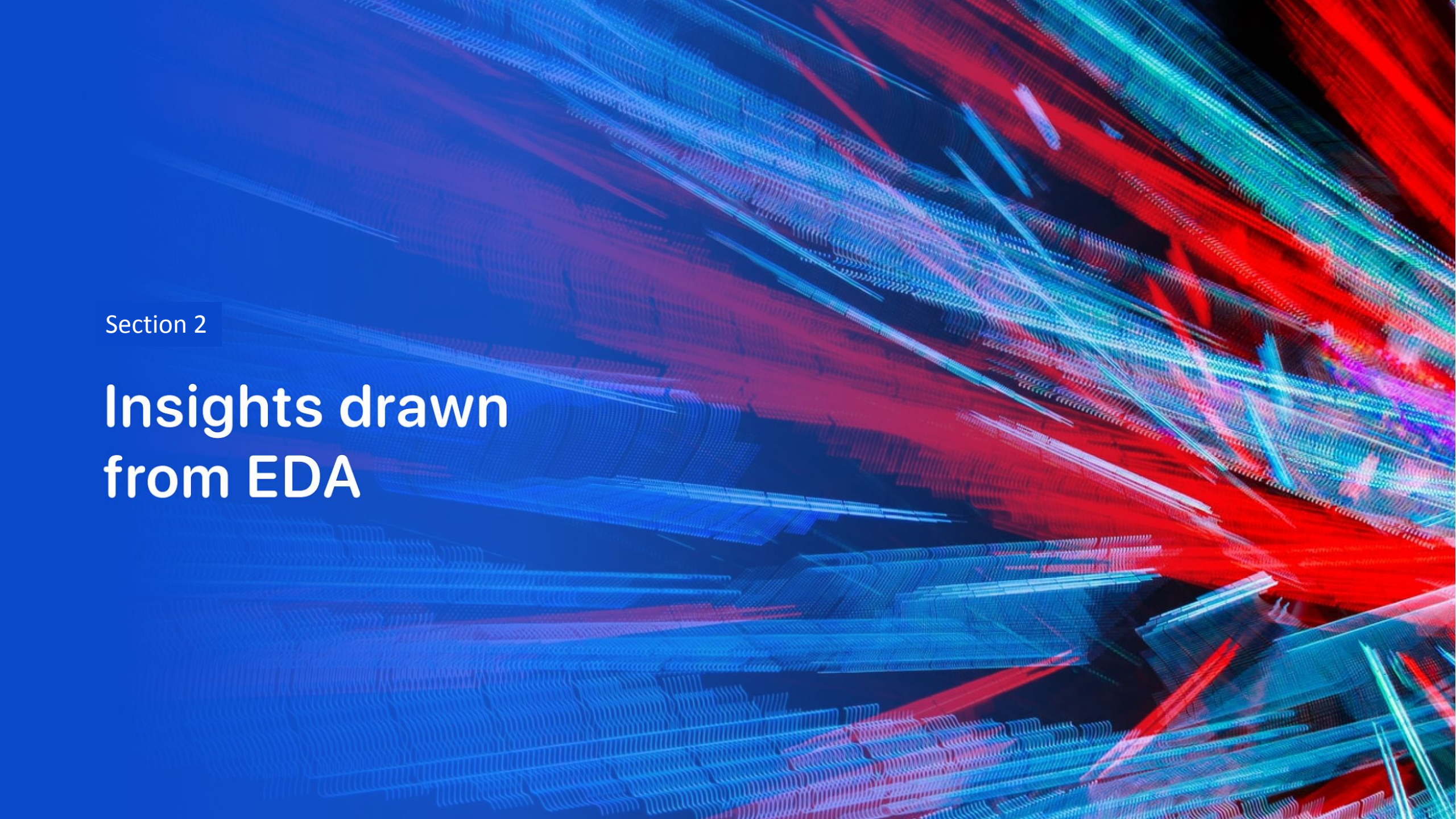
- The goal of this lab was to build and compare four classification models to predict whether SpaceX's Falcon 9 first stage would land successfully. Every model followed the same pipeline: standardise → split → tune with GridSearchCV → evaluate on test data → plot confusion matrix.
- Pre-processing** involved standardising all features using `StandardScaler`, then splitting the data 80/20 into training and test sets (`test_size=0.2`, `random_state=2`). `GridSearchCV` with `cv=10` (10-fold cross-validation) was used across all four models to systematically search hyperparameter grids and select the best configuration without overfitting to the test set.
- The Decision Tree achieved the highest cross-validation score (87.3%) but collapsed to 50% on test data — a clear sign of overfitting despite GridSearchCV tuning. Logistic Regression, SVM, and KNN all generalised consistently at 83.3% test accuracy, making them the better performing models overall. The confusion matrices confirmed that the main error across all models was false positives — predicting a successful landing when the rocket did not land.
- The flowchart traces the full pipeline left to right — pre-processing feeds into the train/test split, which fans out into all four GridSearchCV tuning branches simultaneously, then converges on confusion matrix evaluation and a final verdict. The Decision Tree's dramatic drop from 87.3% CV accuracy to 50% test accuracy stands out as the key finding — high cross-validation scores don't always translate to real-world generalisation, and Logistic Regression, SVM, and KNN proved to be the more reliable models for this task.
- 9.SpaceX_Machine Learning Prediction_Part_5.ipynb



Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Model	Best CV Accuracy	Test Accuracy
Logistic Regression	84.6%	83.3%
SVM	84.8%	83.3%
Decision Tree	87.3%	50.0% ⚠
KNN	84.8%	83.3%

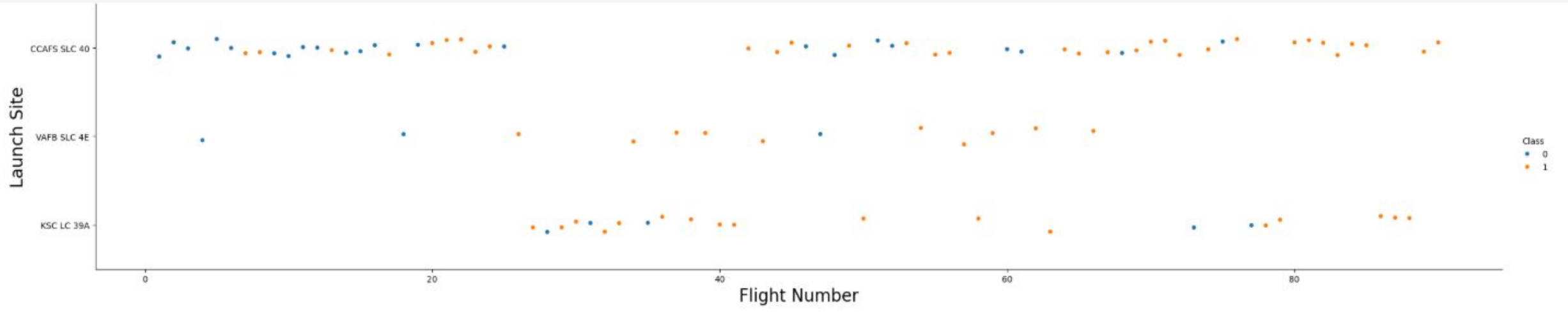


Section 2

Insights drawn from EDA

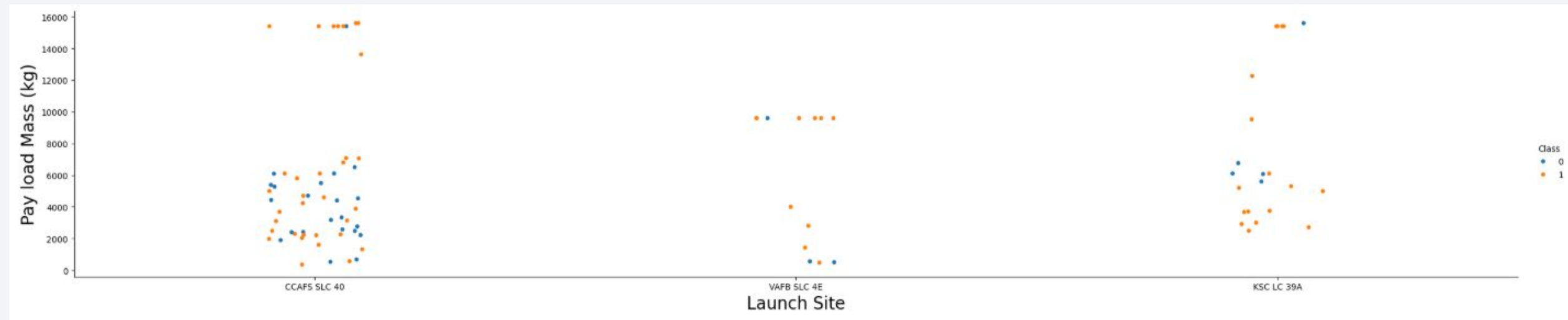
Flight Number vs. Launch Site

- Show a scatter plot of Flight Number vs. Launch Site



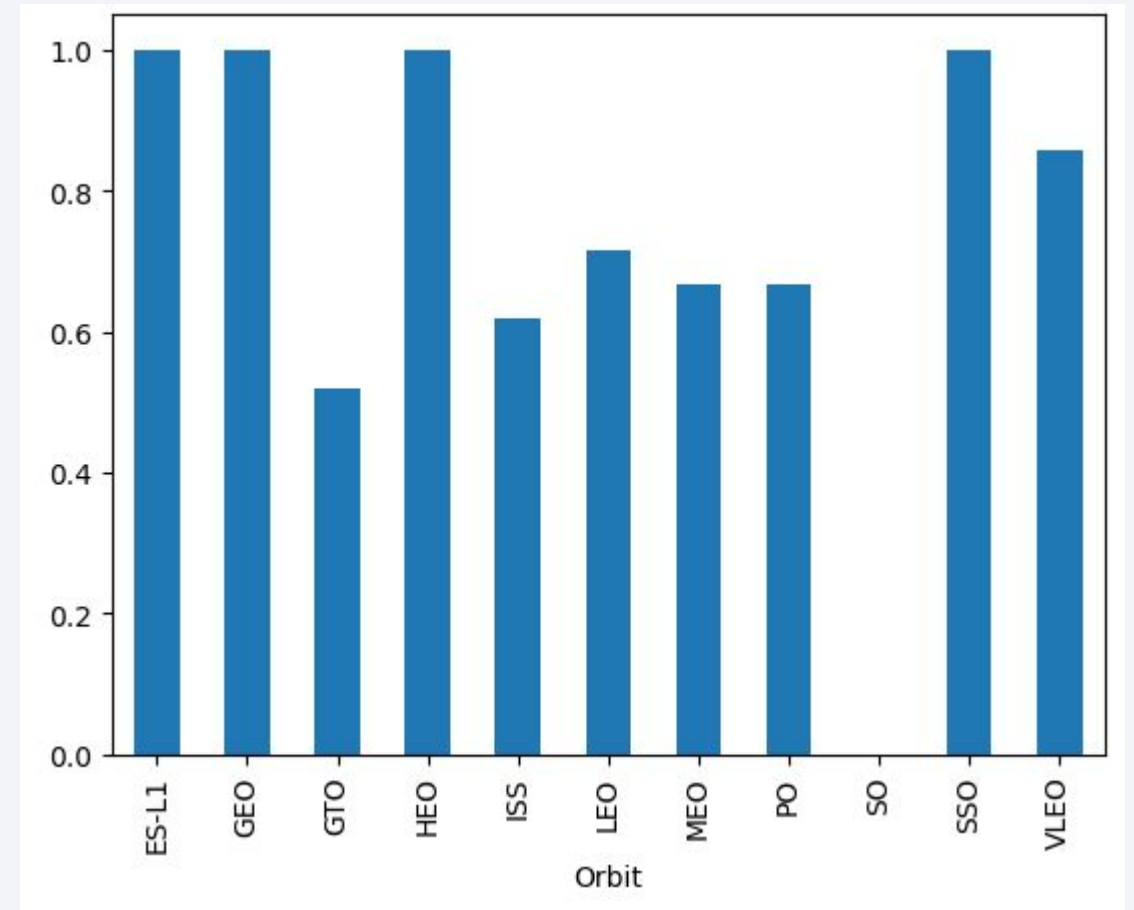
Payload vs. Launch Site

- Show a scatter plot of Payload vs. Launch Site



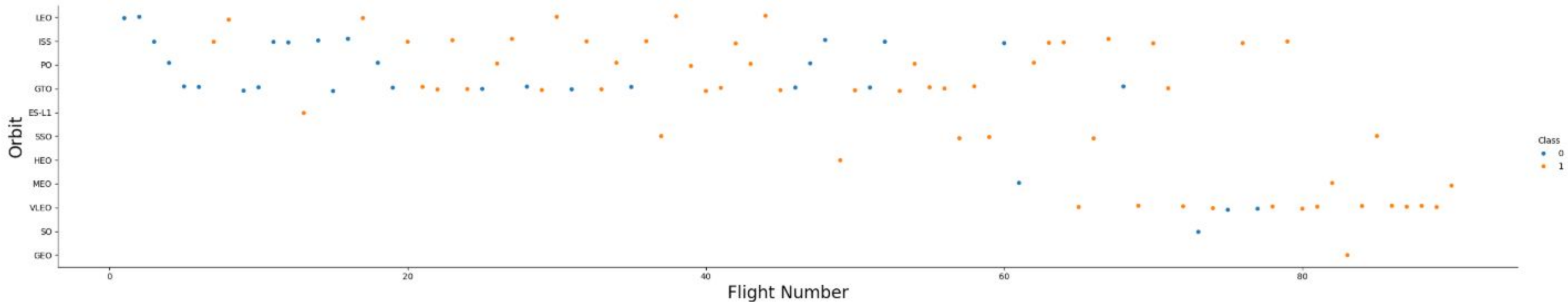
Success Rate vs. Orbit Type

- Show a bar chart for the success rate of each orbit type



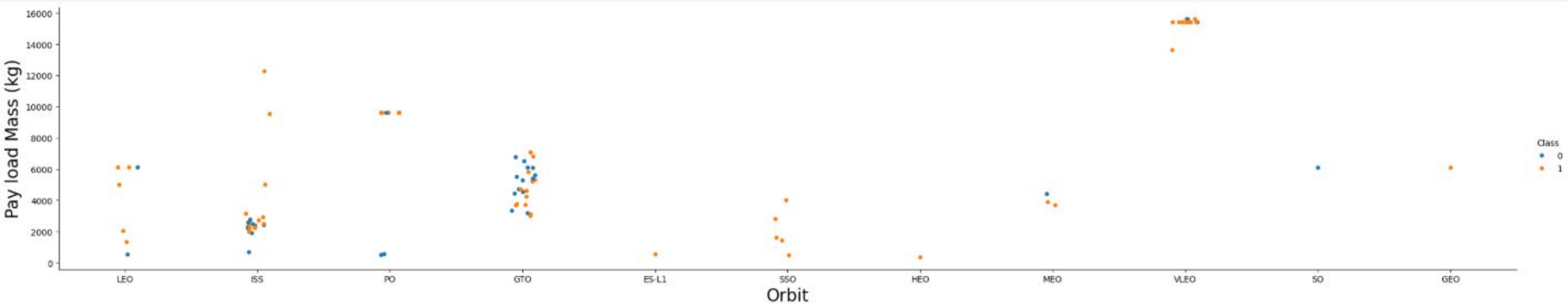
Flight Number vs. Orbit Type

- Show a scatter point of Flight number vs. Orbit type



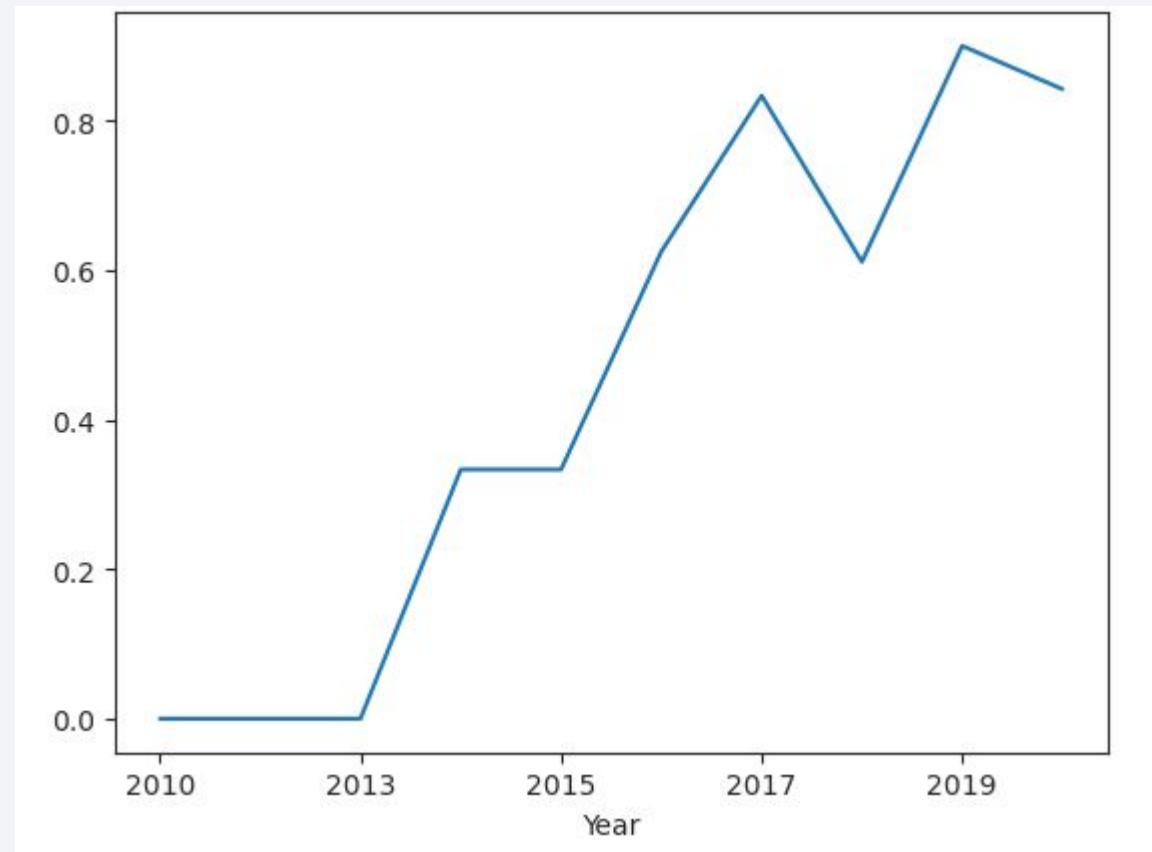
Payload vs. Orbit Type

- Show a scatter point of payload vs. orbit type



Launch Success Yearly Trend

- Show a line chart of yearly average success rate



All Launch Site Names

- Find the names of the unique launch sites

Task 1

Display the names of the unique launch sites in the space mission

```
command = "SELECT * FROM SPACEXTABLE"
cur.execute(command)
rows = cur.fetchall()

# for row in rows:
#     print(row)

# # Get column names
# col_names = [description[0] for description in cur.description]
# print(col_names)

df = pd.read_sql_query("SELECT * FROM SPACEXTABLE", con)

print(df['Launch_Site'].unique())
```

```
['Date', 'Time (UTC)', 'Booster_Version', 'Launch_Site', 'Payload', 'PAYLOAD_MASS__KG_', 'Orbit', 'Customer', 'Mission_Outcome', 'Landing_Outcome']
<StringArray>
['CCAFS LC-40', 'VAFB SLC-4E', 'KSC LC-39A', 'CCAFS SLC-40']
Length: 4, dtype: str
```

Launch Site Names Begin with 'CCA'

- Find 5 records where launch sites begin with 'CCA'

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
df[df['Launch_Site'].str.startswith('CCA')].head()
```

	Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
0	2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
1	2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of...	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2	2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
3	2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
4	2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

- Calculate the total payload carried by boosters from NASA

Task 3

Display the total payload mass carried by boosters launched by NASA (CRS)

```
df['PAYLOAD_MASS_KG_'].sum()
```

```
np.int64(619967)
```

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1

Task 4

Display average payload mass carried by booster version F9 v1.1

```
df['PAYLOAD_MASS_KG_'].mean()
```

```
np.float64(6138.287128712871)
```

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint: Use min function

```
df['Date'].min()
```

```
'2010-06-04'
```

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000

Task 6

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
df1 = df[ df['Mission_Outcome'] == 'Success']  
df2 = df1[df1['PAYLOAD_MASS__KG_'].between(4000,6000)]  
df2.Launch_Site.unique()
```

```
# df['Mission_Outcome'] == 'Success' &
```

```
<StringArray>  
['CCAFS LC-40', 'KSC LC-39A', 'CCAFS SLC-40', 'VAFB SLC-4E']  
Length: 4, dtype: str
```

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcomes

Task 7

List the total number of successful and failure mission outcomes

```
success = df[ df['Mission_Outcome'] == 'Success']  
fail = df1 = df[ df['Mission_Outcome'] != 'Success']  
print(str(len(success)) + " success missions")  
print(str(len(fail)) + " failed missions")
```

```
98 success missions  
3 failed missions
```

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass

Task 8

List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
df_max = df[df['PAYLOAD_MASS_KG_'] == df['PAYLOAD_MASS_KG_'].max()]
df_max.Launch_Site.unique()
```

```
<StringArray>
['CCAFS SLC-40', 'KSC LC-39A']
Length: 2, dtype: str
```

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

Date	Booster_Version	Launch_Site
2015-01-10	F9 v1.1 B1012	CCAFS LC-40
2015-04-14	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order

Landing_Outcome	Landing_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

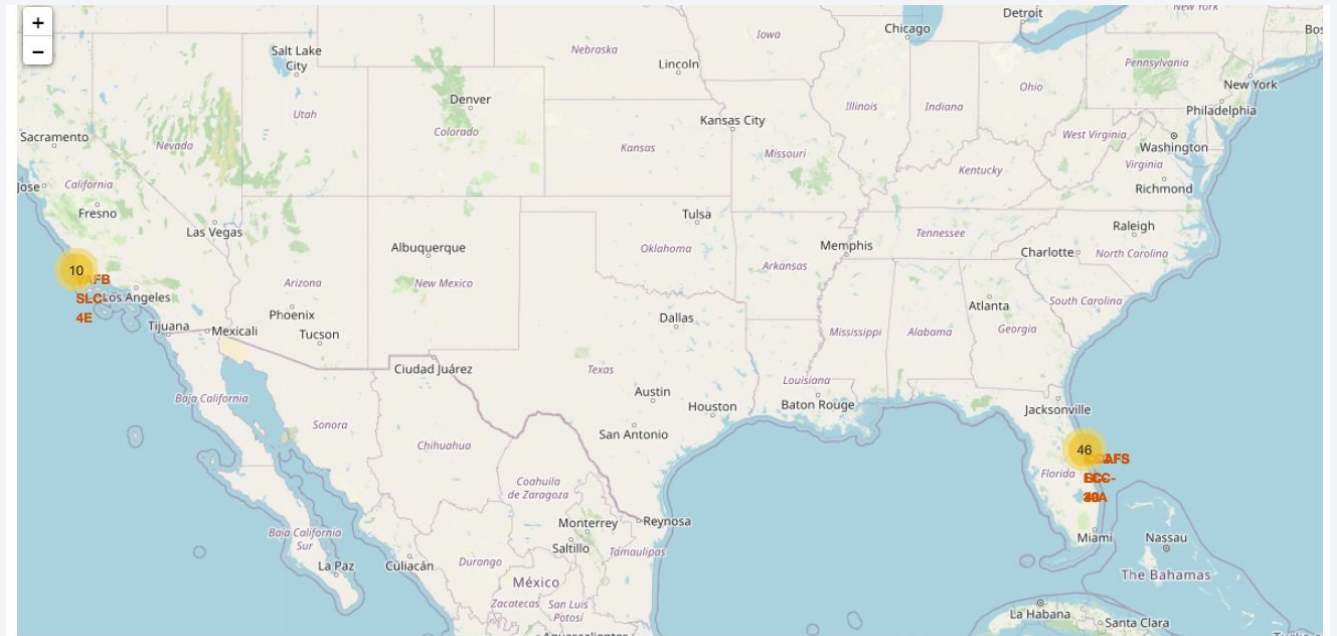
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue background on the left and a satellite photograph of Earth on the right. The Earth's surface is dark, with numerous bright yellow and orange lights representing cities and urban areas. The horizon of the Earth is visible as a thin, curved line separating the dark surface from the deep blue of space.

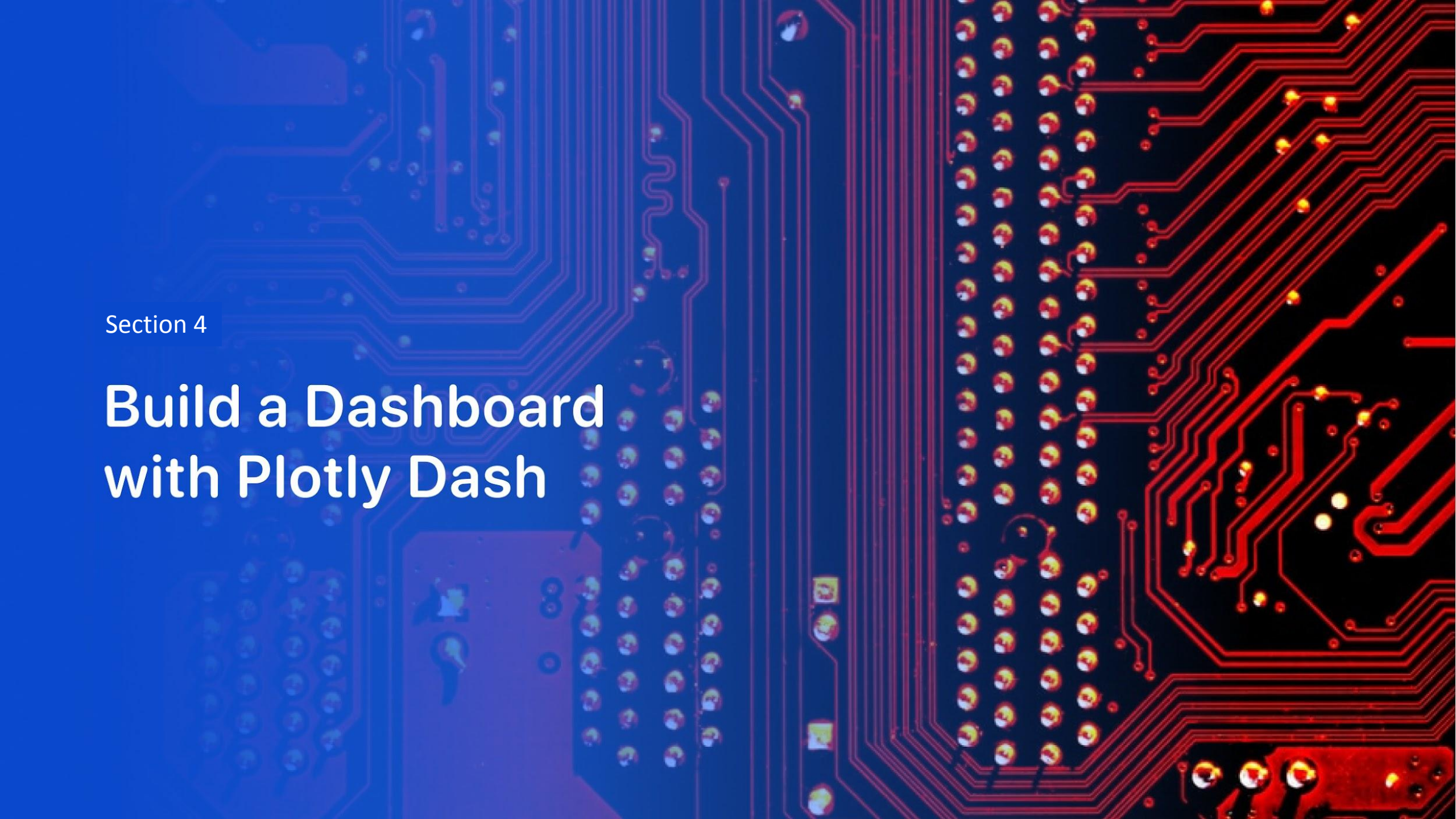
Section 3

Launch Sites Proximities Analysis

Folium visual analytics

- Here's a breakdown of the key elements and findings from the map:
- **The four SpaceX launch sites**
- Three sites cluster tightly together at Cape Canaveral, Florida — KSC LC-39A (Kennedy Space Center), CCAFS LC-40, and CCAFS SLC-40 — all sitting at approximately 28.5°N latitude. The fourth site, VAFB SLC-4E (Vandenberg Air Force Base), is located on the California coast at about 34.6°N. The two clusters are roughly 4,100 km apart.
- **Why these locations matter**
- Two geographic patterns stand out from the lab's guiding questions:
- **Near the Equator, but not on it.** All sites are in the Northern Hemisphere between 28° and 35° North — well short of the equator. However, they are relatively low-latitude by North American standards. Launching from lower latitudes gives rockets a velocity boost from Earth's rotation (Earth spins fastest at the equator), reducing fuel costs for eastward launches into standard orbits. Florida's sites at ~28.5°N offer a meaningful advantage over, say, a site in Canada.
- **All are coastal sites.** Both Florida sites face the Atlantic Ocean to the east, and VAFB faces the Pacific to the west. This is deliberate: rockets launch over open ocean to avoid populated areas in the event of an explosion or debris drop during ascent. The sea provides a natural safety corridor.
- **Florida vs. California — different orbital purposes.** The Florida sites (east-facing) are optimized for eastward launches into equatorial and geosynchronous orbits, maximizing the Earth-rotation velocity boost. VAFB (west-facing, higher latitude) is used for polar and sun-synchronous orbits, which require a southward trajectory along the California/Pacific coast — a trajectory that would be impossible from Florida without flying over populated land.



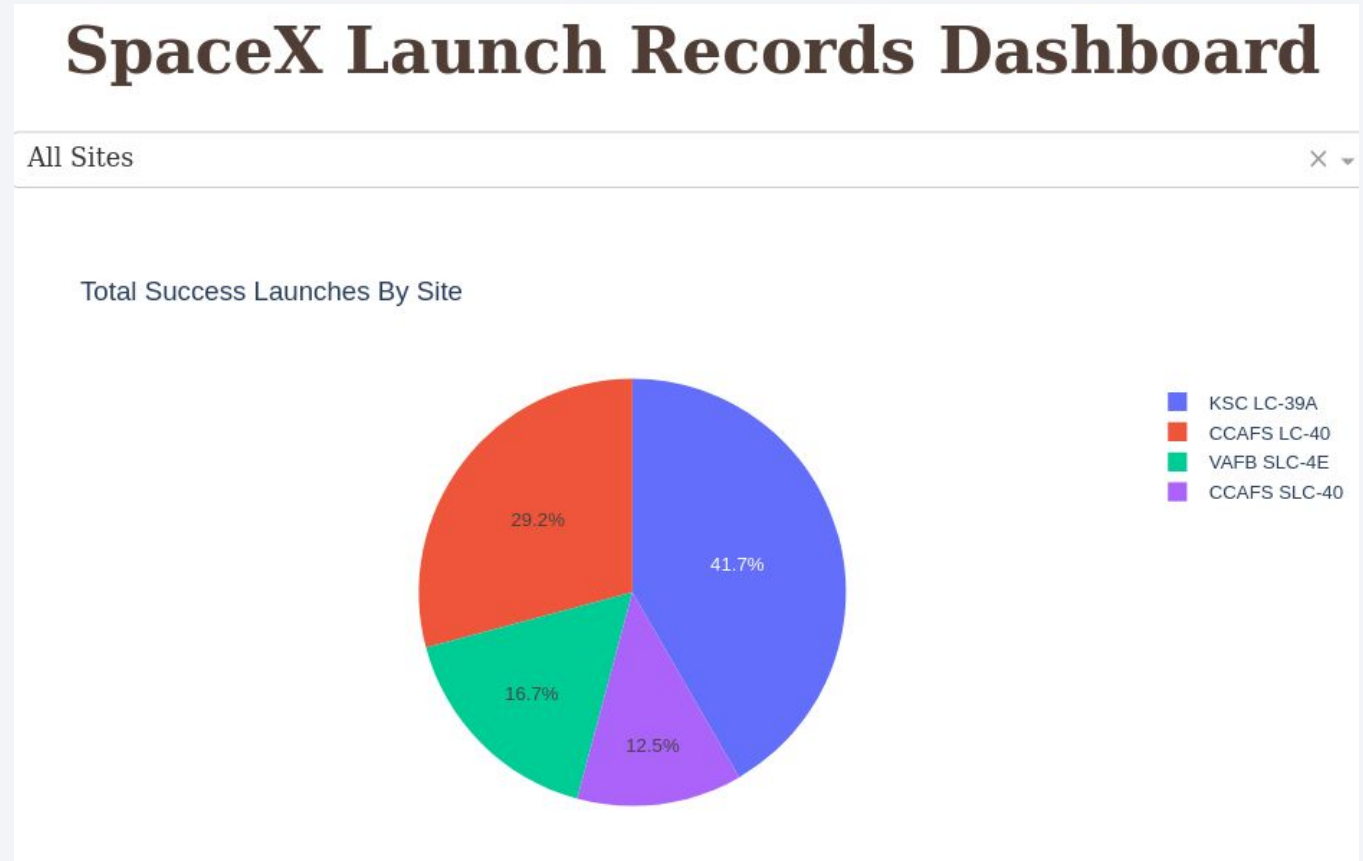


Section 4

Build a Dashboard with Plotly Dash

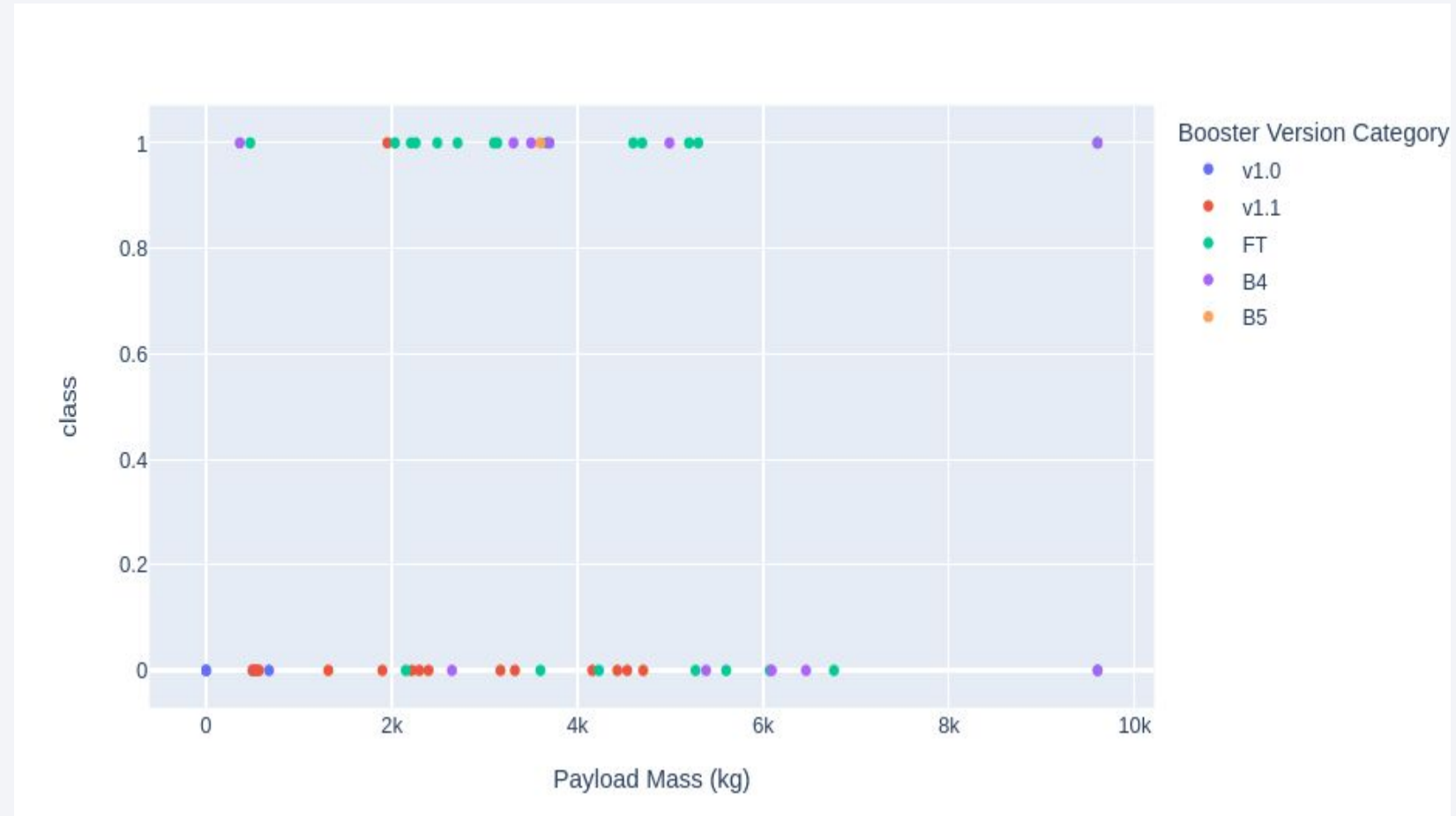
Dashboard Visual Analytics

- KSC has the highest chance of successful launch



Dashboard Visual Analytics

- FT booster version and mid payload mass of 2-6k has the highest success rate

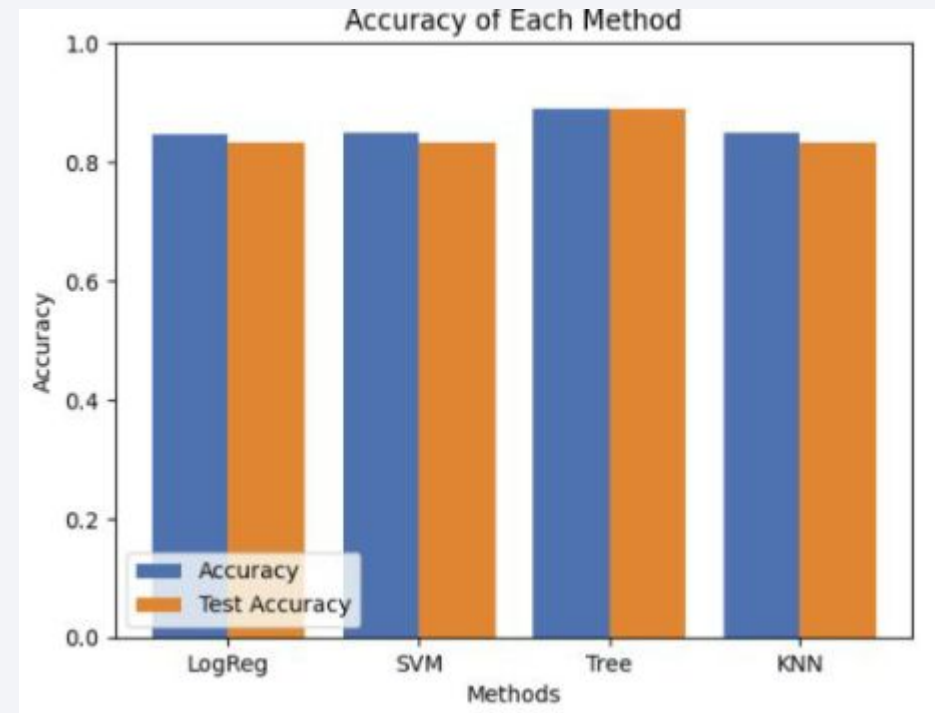


Section 5

Predictive Analysis (Classification)

Classification Accuracy

- Visualize the built model accuracy for all built classification models, in a bar chart
- Decision Tree Classifier has the highest classification accuracy



Confusion Matrix

TASK 5

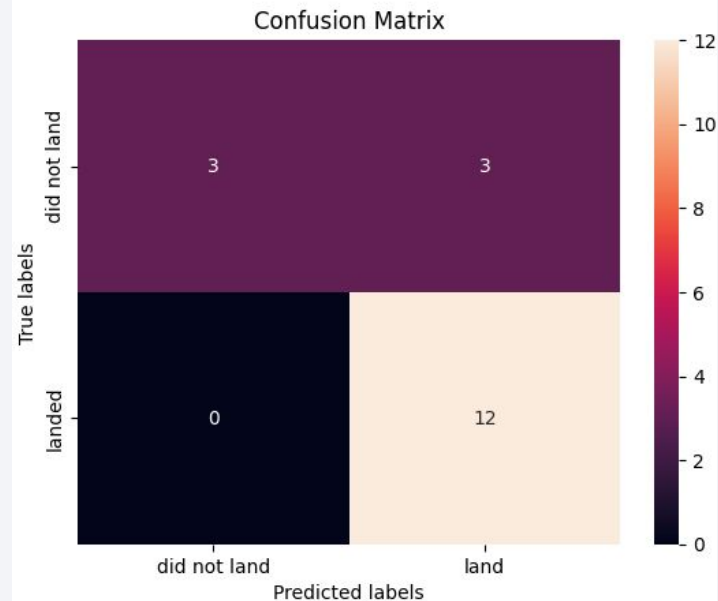
Calculate the accuracy on the test data using the method `score` :

```
accuracy = logreg_cv.score(X_test, Y_test)
print(f"Test Accuracy: {accuracy}")
```

Test Accuracy: 0.8333333333333334

Lets look at the confusion matrix:

```
yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



TASK 7

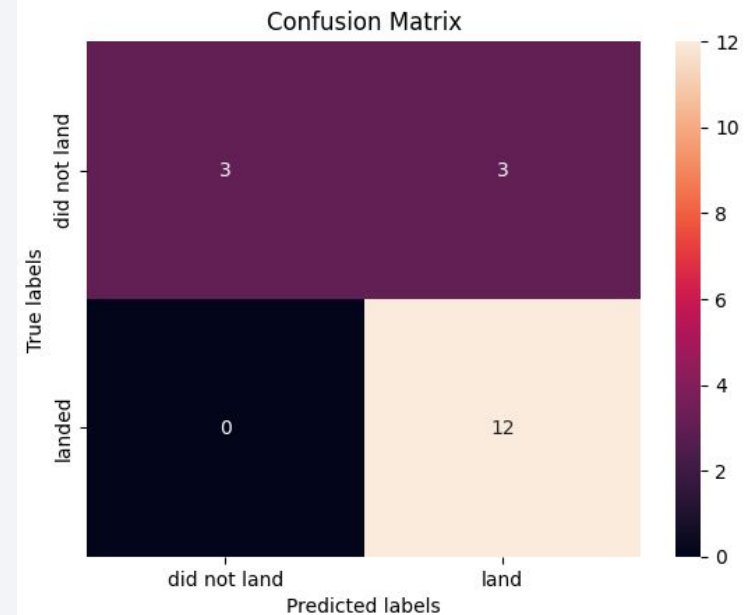
Calculate the accuracy on the test data using the method `score` :

```
accuracy = svm_cv.score(X_test, Y_test)
print(f"Test Accuracy: {accuracy}")
```

Test Accuracy: 0.8333333333333334

We can plot the confusion matrix

```
yhat=svm_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



Confusion Matrix

TASK 9

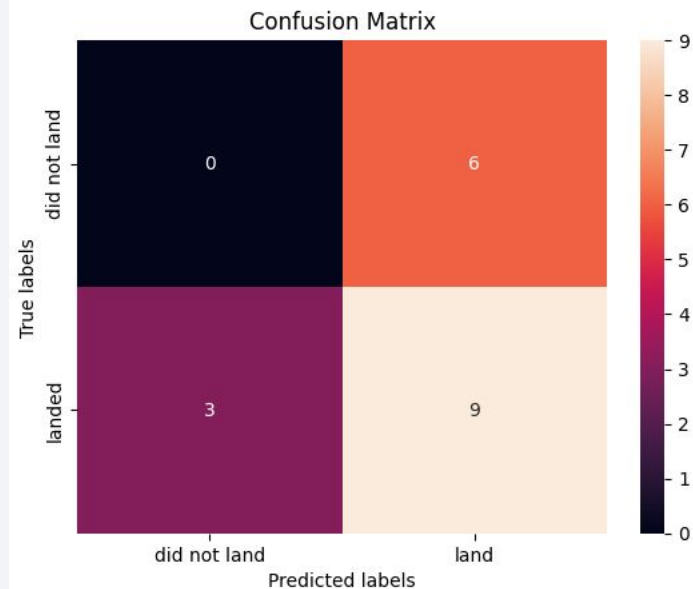
Calculate the accuracy of tree_cv on the test data using the method `score` :

```
accuracy = tree_cv.score(X_test, Y_test)
print(f"Test Accuracy: {accuracy}")
```

Test Accuracy: 0.5

We can plot the confusion matrix

```
yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



TASK 11

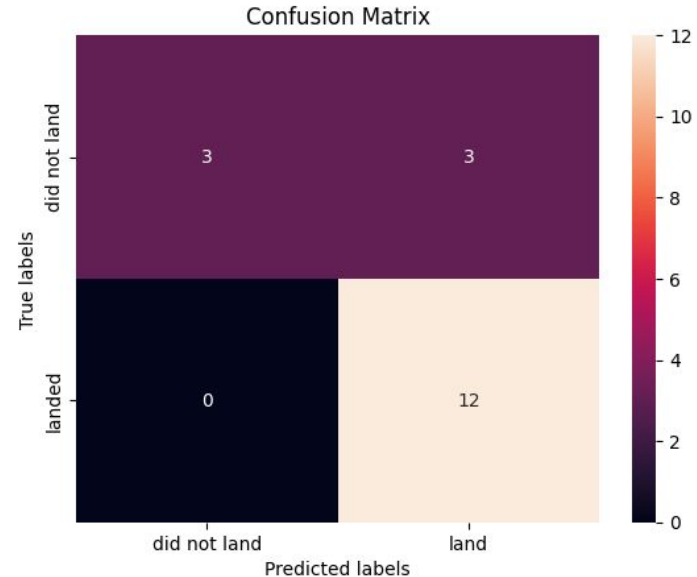
Calculate the accuracy of knn_cv on the test data using the method `score` :

```
accuracy = knn_cv.score(X_test, Y_test)
print(f"Test Accuracy: {accuracy}")
```

Test Accuracy: 0.8333333333333334

We can plot the confusion matrix

```
yhat = knn_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



- 'svm, logreg, knn are equal, decision tree is the worst

Conclusions

- SpaceX selects coastal launch sites to ensure rockets fly over open ocean during ascent, minimizing risk to populated areas, while lower-latitude locations like Cape Canaveral ($\sim 28.5^\circ\text{N}$) provide a natural velocity boost from Earth's rotation that reduces fuel consumption for eastward orbital launches. Through interactive geospatial analysis using Folium, I applied marker clustering, color-coded success/failure indicators, and mouse-position tracking to visualize launch outcomes across all four sites, revealing which locations had relatively higher success rates. Distance calculations between launch sites and surrounding proximities — such as coastlines, highways, and railways — demonstrated how proximity analysis and coordinate-based measurement techniques can uncover meaningful geographical patterns that complement the statistical findings from earlier exploratory data analysis using Matplotlib and Seaborn.

Thank you!

